

Parcial de Python — v3

Condicionales · for/while · def & lambda · Ciencia de Datos

🕒 90 min
📄 20 preguntas
🏆 10 puntos

Instrucciones generales

- ▶ Individual, en computador
- ▶ Solo Python estándar
- ▶ Guarde cada respuesta en un .py
- ▶ Sin conexión a internet
- ▶ Cada pregunta vale **0.5 pts**
- ▶ Se evalúa lógica y casos borde

λ = uso obligatorio de **lambda** | = manejo obligatorio de **casos borde**

I CONDICIONALES · PREGUNTAS 1 – 5

- 1 Cree `categoria_riesgo(p)` con 5 categorías: Muy bajo / Bajo / Medio / Alto / Muy alto según umbrales 0.2, 0.4, 0.6, 0.8.
- 2 Con `predicciones=[0,1,1,0,1,0,0,1]` y `reales=[0,1,0,0,1,1,0,1]`, imprima TP/TN/FP/FN por cada par con un `for`.
- 3 `es_mejora(actual, anterior)`: retorna `True` si mejora, `False` si igual, `"empeoró"` si baja.
- 4 Con `{"precision":0.82,"recall":0.75,"f1":0.78}`, imprima "Modelo balanceado" si todas > 0.75, si no indique cuáles revisar.
- 5 `validar_dato(x)`: retorne "numérico válido", "nulo" o "inválido" (detecte NaN con `x != x`).

II CICLOS FOR · PREGUNTAS 6 – 10

- 6 Con `tiempos=[1.2,0.8,1.5,0.9,1.1,2.3]`, halle el promedio y cuántos superan `prom+0.3`.
- 7 Con `ventas=[120,340,210,450,380,500]`, genere una lista con "alta"/"media"/"baja" según umbrales 400 y 200.
- 8 De una lista de dicts con clave `activo`, sume solo los `valor` de registros activos.
- 9 Transponga `[[1,2,3],[4,5,6]]` → `[[1,4],[2,5],[3,6]]` con `for` anidados.
- 10 Cuente ocurrencias de "data" en `"data science data analysis data"` sin usar `.count()`.

III CICLOS WHILE · PREGUNTAS 11 – 15

- 11 Imprima Fibonacci hasta superar 500 con `while`.
- 12 Sume los dígitos de `987654321` con `while` (sin `str`).
- 13 Entrenamiento: `loss=5.0`, mientras `> 0.1` → `loss = loss*0.9 - 0.05`. Cuente iteraciones.
- 14 Con `[10,20,30,-1,50,60]`, sume con `while` hasta hallar negativo.
- 15 Acumule `random.randint(1,20)` hasta ≥ 100 . Imprima pasos y total.

- λ 16 Cree `rango(lista)` con `def` (sin `max` / `min`, usando `for`) y su versión `rango_1 = lambda ...`. Retornan `None` si está vacía.
- λ 17 Ordene `personas` por **edad ascendente** y **salario descendente** con `lambda + sorted()`.
- λ 18 Con `lambda + filter()`, extraiga valores en `[0.4, 0.9]` de `[0.15, 0.85, 0.42, 0.91, 0.33, 0.67]`.
- λ 19 Con `lambda + map()`, transforme `[(1,2), (3,4), (5,6)]` en `[3,7,11]`.
- λ 20 Cree `crear_pipeline(umbral, factor)` que retorne una `lambda` que multiplique por `factor` y devuelva "aceptado" / "rechazado".

Rúbrica y Soluciones — v3

Documento para el docente · uso interno

 Confidencial



RÚBRICA DE EVALUACIÓN

CRITERIO	PUNTOS
Sintaxis correcta y uso adecuado de estructuras	0.2
Lógica correcta del algoritmo	0.2
Manejo de casos borde (listas vacías, división por cero, NaN)	0.1
Total por pregunta	0.5



SOLUCIONES BREVES (REFERENCIA)

```
# 1
def categoria_riesgo(p):
    if p < 0.2: return "Muy bajo"
    elif p < 0.4: return "Bajo"
    elif p < 0.6: return "Medio"
    elif p < 0.8: return "Alto"
    else: return "Muy alto"

# 2
pred = [0,1,1,0,1,0,0,1]
real = [0,1,0,0,1,1,0,1]
for p, r in zip(pred, real):
    if p==1 and r==1: print("TP")
    elif p==0 and r==0: print("TN")
    elif p==1 and r==0: print("FP")
    else: print("FN")

# 3
def es_mejora(actual, anterior):
    if actual > anterior: return True
    elif actual == anterior: return False
    else: return "empeoró"

# 4
m = {"precision":0.82,"recall":0.75,"f1":0.78}
if all(v > 0.75 for v in m.values()):
    print("Modelo balanceado")
else:
    for k, v in m.items():
        if v <= 0.75: print(f"Revisar {k}")

# 5
def validar_dato(x):
    if x is None: return "nulo"
    if isinstance(x, bool): return "inválido"
    if isinstance(x, (int, float)):
        if x != x: return "inválido"
        return "numérico válido"
    return "inválido"

# 6
t = [1.2,0.8,1.5,0.9,1.1,2.3]
p = sum(t) / len(t)
c = 0
for x in t:
    if x > p + 0.3: c += 1
print(p, c)
```

```
# 7
v = [120,340,210,450,380,500]
cat = []
for x in v:
    if x >= 400: cat.append("alta")
    elif x >= 200: cat.append("media")
    else: cat.append("baja")

# 8
regs = [
    {"id":1,"valor":10,"activo":True},
    {"id":2,"valor":25,"activo":False},
    {"id":3,"valor":40,"activo":True},
    {"id":4,"valor":15,"activo":True},
]
s = 0
for r in regs:
    if r["activo"]: s += r["valor"]

# 9
m = [[1,2,3],[4,5,6]]
tr = []
for j in range(len(m[0])):
    fila = []
    for i in range(len(m)):
        fila.append(m[i][j])
    tr.append(fila)

# 11
a, b = 0, 1
while b <= 500:
    print(b); a, b = b, a + b

# 12
n, s = 987654321, 0
while n > 0:
    s += n % 10; n //= 10
print(s)

# 13
loss, it = 5.0, 0
while loss > 0.1:
    loss = loss * 0.9 - 0.05
    it += 1
print(it, loss)

# 14
v = [10,20,30,-1,50,60]
i, s = 0, 0
```

```

while i < len(v) and v[i]>= 0:
    s += v[i]; i += 1
print(s)

# 15
import random
obj, acc, p = 100, 0, 0
while acc < obj:
    acc += random.randint(1, 20); p += 1
print(p, acc)

# 16
def rango(lista):
    if not lista: return None
    may = men = lista[0]
    for x in lista:
        if x > may: may = x
        if x < men: men = x
    return may - men

rango_1 = lambda l: None if not l else max(l) - min(l)

# 17
personas = [
    {"nombre": "Ana", "edad": 30, "salario": 2500},
    {"nombre": "Luis", "edad": 25, "salario": 3000},
    {"nombre": "Marta", "edad": 30, "salario": 3200},
    {"nombre": "Pedro", "edad": 25, "salario": 2800},
]
orden = sorted(personas,
    key=lambda p: (p["edad"], -p["salario"]))

# 18
f = list(filter(lambda x: 0.4 <= x <= 0.9,
    [0.15, 0.85, 0.42, 0.91, 0.33, 0.67]))

# 19
s = list(map(lambda t: t[0] + t[1],
    [(1, 2), (3, 4), (5, 6)]))

# 20
def crear_pipeline(umbral, factor):
    return lambda v: "aceptado" if v * factor > umbral else "rechazado"

pipe = crear_pipeline(0.6, 1.5)
print(pipe(0.5)) # aceptado
print(pipe(0.3)) # rechazado

```